1.1.1 The FRACTRAN Threshold: Algebraic and Modular Minima for Universal Computation

Introduction to the Arithmetic Limits of Computability

The theoretical boundary of computational universality has remained a central preoccupation of mathematical logic and theoretical computer science since Alan Turing's seminal 1936 paper, "On Computable Numbers, with an Application to the Entscheidungsproblem".[1] Turing demonstrated that it is possible to construct a single, universal machine capable of computing any computable sequence by operating on a finite number of internal conditions, or "m-configurations," and an infinite memory tape.[1] While Turing's conceptualization fundamentally shaped the development of the stored-program computer and influenced John von Neumann's architecture for the EDVAC [1], subsequent generations of mathematicians have sought to strip the universal computing machine of its mechanical metaphors. The objective has been to discover the absolute lowest threshold of syntactical and algebraic complexity required to sustain Turing-complete logic.

Among the most elegant and mathematically profound discoveries in this domain is FRACTRAN, an esoteric and minimalist programming language devised by John Horton Conway in 1987.[2] Conway, a prolific mathematician renowned for his contributions to finite group theory, knot theory, combinatorial game theory, and cellular automata [5], introduced FRACTRAN as an arithmetic distillation of computation.[2] FRACTRAN abandons the concept of a tape, a read/write head, and explicit state transition tables. Instead, it relies entirely on the multiplicative structure of the rational numbers.[7]

A FRACTRAN program is defined as an ordered, finite list of positive rational numbers (fractions) $f_1, f_2, \ldots, f_k$.[2] The global state of the entire computational system is encoded within a single positive integer N.[9] The execution of the program follows a rigidly deterministic algorithm: at any given discrete time step t, the machine multiplies the current state integer N_t by the earliest fraction f_i in the ordered sequence for which the resulting product $f_i N_t$ is an integer.[2] If such a fraction exists, the system updates to the new state $N_{t+1} = f_i N_t$, and the cycle repeats. If there is no fraction in the list that produces an integer when multiplied by the current state, the execution halts.[2]

Despite its staggering simplicity—boasting a syntax that can be learned in seconds

[10]—FRACTRAN is completely computationally universal. It can simulate any arbitrary Turing machine, generate transcendental numbers, and compute prime sequences.[2] Conway famously demonstrated its power through specific programs: a 14-fraction sequence called PRIMEGAME that generates the prime numbers sequentially as exponents of 2, and a highly inefficient 38-fraction sequence called PIGAME that computes the decimal expansion of π utilizing Wallis' product.[2] Furthermore, Conway outlined a universal FRACTRAN machine capable of simulating any other machine.[10]

The "FRACTRAN Threshold" refers to the pursuit of the absolute theoretical mathematical minimums required to encode a Universal Turing Machine (UTM) within this fractional framework. This threshold is fundamentally parameterized by two distinct metric spaces. The first is the modular dimension: what is the absolute minimum number of distinct residue classes modulo d (where d is the least common multiple of all fractional denominators) necessary to encode Turing-complete control logic?[3] The second is the algebraic dimension: what is the minimum prime signature required for the fractional coefficients a_i to sustain universal state transitions without introducing chaotic algebraic diffusion?[11]

By rigorously mapping the formal limits of small Universal Register Machines—particularly those pioneered by Ivan Korec and Ludmila Gregušová [12]—onto the purely arithmetic geometry of FRACTRAN, we can establish exact mathematical bounds on these thresholds. This analysis explores the intersection of Minsky machines, prime signature lattices, Collatz-like piecewise affine maps, and undecidability theorems to isolate the absolute boundary of universal computation.[4]

The Formal Syntax and Arithmetic State Space

To comprehensively evaluate the minimal thresholds of FRACTRAN, it is first necessary to formalize its state space and operational mechanics. FRACTRAN operates on a paradigm that bridges continuous arithmetic and discrete state logic by exploiting the Fundamental Theorem of Arithmetic.[7] This theorem guarantees that every positive integer greater than 1 can be represented as a unique product of prime numbers.[11] Consequently, the single integer N that represents the global state of a FRACTRAN machine can be mapped to a vector in an infinite-dimensional lattice.

Let $\mathbb{P} = \{p_1, p_2, p_3, \dots\} = \{2, 3, 5, 7, 11, \dots\}$ denote the well-ordered infinite set of prime numbers. The state integer N is uniquely expressed through its prime factorization:

$$N = p_1^{r_1} p_2^{r_2} p_3^{r_3} \cdots = \prod_{j=1}^{\infty} p_j^{r_j}$$

where each exponent $r_j \in \mathbb{N}_0$, and only a finite number of these exponents are strictly greater than zero.[7]

In this mathematical framework, FRACTRAN functions strictly as a Register Machine or Counter Machine. The continuous integer N is merely a Gödel-numbered vessel. The actual computational data—the values of the individual registers—are stored within the exponents r_j of the corresponding prime bases p_j.[7] For example, a system with three active registers containing the values a, b, and c would be encoded into the integer state $N = 2^a 3^b 5^c$.[8]

The fractions that comprise a FRACTRAN program represent fixed state transition operators that simultaneously query, decrement, and increment these prime registers. Consider a general fraction $f_i = \frac{A_i}{B_i}$ located at the i-th position in the FRACTRAN sequence. The numerator and denominator can be similarly factorized into their prime components:

$$A_i = \prod_{j=1}^{\infty} p_j^{a_{ij}}, \quad B_i = \prod_{j=1}^{\infty} p_j^{b_{ij}}$$

When the FRACTRAN interpreter evaluates the operation $f_i N$, it is essentially verifying a divisibility condition: does the denominator B_i divide the current state N? In the language of register machines, $B_i \mid N$ is mathematically equivalent to checking if the current register state contains sufficient values to satisfy the denominator's prime signature. Specifically, the fraction is "active" if and only if $r_j \geq b_{ij}$ for all j.[7]

If this boolean condition evaluates to true, the fraction is applied, and the state undergoes an atomic algebraic transformation:

$$N_{t+1} = N_t \times \frac{A_i}{B_i} \implies r_j^{(t+1)} = r_j^{(t)} - b_{ij} + a_{ij}$$

This operation fundamentally subtracts the prime signature of the denominator from the global state vector and adds the prime signature of the numerator.[7]

The deterministic execution of FRACTRAN is enforced by the strict linear ordering of the

fractions. The machine evaluates f_1, then f_2, and so forth, returning to f_1 immediately upon any successful integer multiplication.[2] This ordered evaluation functions as a priority-encoded, deeply nested IF-THEN-ELSE logical cascade.[9] Because FRACTRAN completely ignores the additive structure of the integers and operates exclusively on their multiplicative structure, it achieves universality through the isolation of these prime exponents.[7] The absolute minimum limits for a universal FRACTRAN machine must therefore map directly to the smallest known Turing-complete finite automata that can operate under these rigid algebraic constraints.

Universal Register Machines and Minsky's Reductions

To bound the FRACTRAN threshold, we must establish the exact algebraic morphisms between FRACTRAN and classical Universal Register Machines. In 1967, Marvin Minsky published seminal research demonstrating that a register machine equipped with only two unbounded registers is Turing-complete, provided the machine has access to a finite state control.[12] A Universal Minsky Machine takes as its input a Gödel-numbered description of an arbitrary Turing machine, alongside its input tape data, and faithfully emulates its execution.[12]

A standard Minsky machine operates using only two fundamental, atomic instructions:

1. The Increment Operation INC(r, s_next): Unconditionally add 1 to register r and transition the machine to state s_{next}.
2. The Conditional Decrement Operation JZDEC(r, s_true, s_false): Query register r. If $r > 0$, subtract 1 from r and transition to state s_{true}. If $r = 0$, leave the register unchanged and transition to state s_{false}.

By mapping the finite states to distinct prime numbers (acting as the program counter) and the two data registers to a separate pair of distinct prime numbers (acting as the memory vectors), any arbitrary Minsky machine can be explicitly compiled into a FRACTRAN program.[7] Let the Minsky machine states $S_1, S_2, \ldots, S_m$ be represented by the prime indicators $p_{s_1}, p_{s_2}, \ldots, p_{s_m}$. Let the two active memory registers R_1 and R_2 be represented by the primes p_{r_1} and p_{r_2}.

The compilation of a Minsky machine into FRACTRAN yields a highly specific fractional geometry. The following table illustrates the exact arithmetic mappings required to translate discrete control logic into FRACTRAN operators.

Minsky Machine Instruction	**Logical Register Operation**	**FRACTRAN Fraction Encoding**
INC(R1, S_next)	Add 1 to R_1, transition $S_{current} \to S_{next}$	$\frac{p_{r_1} \cdot p_{s_{next}}}{p_{s_{current}}}$
JZDEC(R1, S_T, S_F)	If $R_1 > 0$, decrement R_1, $S_{current} \to S_T$	$\frac{p_{s_T}}{p_{r_1} \cdot p_{s_{current}}}$ (Positioned first in sequence)
JZDEC(R1, S_T, S_F)	If $R_1 = 0$, leave R_1, $S_{current} \to S_F$	$\frac{p_{s_F}}{p_{s_{current}}}$ (Positioned second in sequence)

This direct mathematical compilation proves that the complexity of a universal FRACTRAN program is strictly and inextricably bounded by the size of the minimal universal register machine.[7] If one can locate the absolute smallest universal Minsky machine, one can directly compute the absolute minimum threshold for a FRACTRAN machine by converting its rule set into rational fractions.

Historically, small universal Minsky machines have been subjected to aggressive optimization to find the absolute minimal instruction set. In 1979, Ludmila Gregušová and Ivan Korec published "Small Universal Minsky Machines," presenting a universal machine capable of computing any computable sequence using exactly 37 rules.[12] This marked a significant reduction in the known spatial complexity of universal automata.

Korec continued to optimize these models, culminating in a landmark 1996 paper, "Small universal register machines," published in Theoretical Computer Science.[13] In this paper, Korec detailed the construction of several minimal register machines, systematically driving the number of required operational instructions down from 32, to 22, and ultimately to a staggering minimum of 14 instructions.[13] Korec achieved these reductions through highly sophisticated coding functions for finite sequences of positive integers and the application of strong classical results concerning the distribution of primes.[20]

Conway's own manually optimized universal FRACTRAN machine operates with 21 fractions.[7] However, projecting Korec's 14-instruction universal register machine into the FRACTRAN syntax provides the rigorous mathematical floor. A complete, unsimulated universal FRACTRAN machine can be achieved with a fraction count directly corresponding to these 14 discrete logical states. This reduction establishes the explicit bounds for both the modulus d and the subsequent residue classes.

The Modulo d Residue Class Threshold

The execution of any FRACTRAN program is inherently and entirely a process of evaluating modular arithmetic.[21] For a FRACTRAN program $P = (f_1, f_2, \ldots, f_k)$ populated with fractions $f_i = \frac{A_i}{B_i}$, the operation $f_i N \in \mathbb{N}$ depends strictly on whether the denominator B_i divides the integer N.

Let d be defined as the least common multiple of all denominators present in the FRACTRAN program:

$$d = \operatorname{lcm}(B_1, B_2, \ldots, B_k)$$

Because $B_i \mid N$ if and only if $N \equiv 0 \pmod{B_i}$, the determination of which fraction is actively applied to the state integer is entirely resolved by evaluating the residue class of N modulo d.[21] In number theory, the residue classes of a function modulo n represent all possible values of the remainder.[21] A complete residue system is a set of integers containing exactly one representative element from each equivalence class.[21]

Consequently, any FRACTRAN program is mathematically equivalent to a piecewise linear map $T : \mathbb{N} \to \mathbb{N}$ defined exclusively on the residue classes modulo d:

$$T(N) = c_m N \quad \text{if } N \equiv m \pmod{d}$$

where c_m is the specific rational fraction activated for the residue class m.[3]

The "FRACTRAN Threshold" with respect to the absolute minimum number of distinct residue classes modulo d requires identifying the smallest possible number of modular branches necessary to sustain universal, Turing-complete logic.

If we examine Ivan Korec's minimal universal register machine with exactly 14 instructions [13], we can deduce the structural limits of the modulus d. In the Minsky-to-FRACTRAN compilation scheme, each instruction generates either one fraction (for an unconditional INC) or at most two fractions (for a JZDEC instruction, which requires a successful decrement fraction followed immediately by a zero-branch check).[7] While a 14-instruction machine might naively compile to a FRACTRAN program with up to 28 fractions, many of these instructions share logical states or act unconditionally. Thus, the actual number of mathematically unique denominators is tightly constrained by the state transitions.

When establishing the absolute minimum number of distinct residue classes modulo d, we are explicitly concerned with the number of unique prime denominator signatures that trigger unique computational actions. In a minimal 2-counter machine compiled to FRACTRAN, the denominators are exclusively composed of combinations of the state primes and the two register primes. If the machine utilizes a finite set of states S and 2 registers, the theoretical maximum number of distinct denominators is bounded by $S \times 3$, since any given state can query the first register, the second register, or operate unconditionally without querying either.

However, the threshold for distinct residue classes modulo d that must trigger distinct affine transformations c_m is exactly equal to the number of irreducible functional execution paths in the universal machine. For Korec's 14-instruction register machine [13], there are strictly 14 irreducible algebraic operations that must be conditionally applied to maintain the universal simulation. Therefore, the absolute minimum number of distinct residue classes modulo d required to successfully encode a universal Turing machine is exactly 14.

While the total arithmetic value of d may be extraordinarily large—since d is the product of multiple prime numbers and can grow rapidly with the state space—the number of *distinct active equivalence classes* can be mathematically collapsed into 14 functional branches.[4] We define an active equivalence class as a grouping of residues that trigger the exact same fractional multiplication. Any arithmetic system possessing fewer than 14 distinct conditional branches cannot support the full matrix of Turing-complete logic without requiring external tape memory or falling into unresolvable halting paradoxes.

It is critical to address apparent anomalies in the literature, such as the widely cited 3-fraction sequence $\{\frac{6}{1}, \frac{1}{2}, \frac{1}{3}\}$, which Minsky himself noted for its complex behavior.[13] While this system exhibits highly complex, pseudo-random trajectories and Turing-like state fluctuations, rigorous mathematical analysis has demonstrated that this specific collection "allows for non-trivial loops and does not induce an effective UTM".[23] It lacks the definitive halting states, the structured data

preservation, and the unambiguous conditional branching required for full, unsimulated universality. Thus, while systems with extremely small residue counts can exhibit computational complexity, the absolute minimum threshold for a strict, structurally sound Universal Turing Machine encoding requires 14 distinct modular partitions.[13]

The Algebraic Threshold: Prime Signatures and Square-Free Coefficients

The second fundamental parameter of the FRACTRAN threshold is the minimum prime signature required for the rational coefficients a_i. In classical number theory, the prime signature of an integer is defined as the multiset of the exponents of its prime factorization, conventionally arranged in non-increasing order.[11] Within the context of FRACTRAN, we extend this definition to the rational fractions $f_i = p_1^{v_1} p_2^{v_2} \cdots p_n^{v_n}$, where the prime signature is the formal vector of integer exponents $\mathbf{v} = (v_1, v_2, \ldots, v_n)$, with $v_i \in \mathbb{Z}$.

To ascertain the absolute minimum prime signature required to encode a Universal Turing Machine, we must analyze the atomic operations of the compiled FRACTRAN program. As established in the mapping matrix, the fundamental transitions dictate the following exponent bounds:

- The Increment (INC) operation utilizes the fraction $\frac{p_{r_1} p_{s_{next}}}{p_{s_{current}}}$. The prime signature vector for this fraction contains exactly two $+1$ exponents and exactly one -1 exponent. All other exponents are 0.
- The Decrement (DEC) successful branch utilizes the fraction $\frac{p_{s_{next}}}{p_{r_1} p_{s_{current}}}$. The prime signature vector for this fraction contains exactly one $+1$ exponent and exactly two -1 exponents.
- The Zero-Branch (JUMP) operation utilizes the fraction $\frac{p_{s_{next}}}{p_{s_{current}}}$. The prime signature vector for this fraction contains exactly one $+1$ exponent and exactly one -1 exponent.

The magnitude of the exponents in the minimal prime signature is strictly and absolutely bounded to the set $\{-1, 0, 1\}$. No fractional coefficient within a minimally compiled Universal FRACTRAN machine requires an exponent with an absolute value greater than 1. This establishes what can be formally described as the $(1, 1, 1)$ prime signature threshold: universal computation can be achieved entirely and exclusively through square-free numerators and square-free denominators.[11]

Furthermore, we must quantify the minimum dimension of this signature vector, which corresponds to the minimum number of *distinct primes* required to support universality. Minsky's foundational theorem proves that exactly 2 counters are necessary and sufficient for universality.[12] Thus, 2 distinct primes are irrevocably required for the data registers (commonly designated as 2 and 3).

The finite state control necessitates a program counter. If the program counter were to be compressed and stored as the exponent of a single, third prime (e.g., storing state S as 5^S), then the FRACTRAN fractions would require higher-order polynomial exponents to transition between non-adjacent states. For example, transitioning from state S to $S+4$ would require multiplying the state integer by 5^4, introducing the fraction $\frac{5^4}{1}$, which severely violates the $(1,1,1$ square-free threshold limit.

To rigorously maintain the $(1,1,1$ prime signature bound and ensure strictly affine single-step transitions, the finite state control must be flattened. It must be encoded using distinct primes for each individual state, a paradigm analogous to One-Hot Encoding in digital circuit design. Because Ivan Korec's 14-instruction machine represents the absolute mathematical limit of finite control for a 2-counter machine [20], the state space requires exactly 14 distinct primes. Combined with the 2 register primes, the minimal prime signature dimension required to compile a universal Turing machine into FRACTRAN without violating the square-free coefficient bound is 16 distinct primes.

While highly convoluted encoding schemes can compress the prime dimension down to 3 distinct primes by abandoning the linear state mapping and utilizing highly complex, non-square-free exponents, doing so forces the FRACTRAN machine out of the $(1,1,1$ threshold. The strict geometric threshold balancing minimum exponent magnitude against minimum modular complexity is solidly fixed at square-free coefficients distributed across 16 prime dimensions.[11]

Collatz-Like Dynamics, Piecewise Affine Maps, and Topological Non-Mixing

The deep, underlying mathematical connection between FRACTRAN and the Collatz $3x+1$ conjecture sheds immense light on the nature of the residue class threshold.[12] Both computational systems are fundamentally defined by piecewise linear affine operations executed on residue classes modulo d.

The classical Collatz function is defined as a piecewise affine map modulo 2:

$$C(n) = \begin{cases} \frac{n}{2} & \text{if } n \equiv 0 \pmod 2 \\ 3n+1 & \text{if } n \equiv 1 \pmod 2 \end{cases}$$

In 1972, John Conway published a landmark paper titled "Unpredictable Iterations," in which he proved that a natural mathematical generalization of the Collatz function to moduli $d > 2$ is formally undecidable.[4] Conway demonstrated that there exists no general algorithm capable of determining whether a given integer N, when subjected to a generalized Collatz-like function, will eventually reach the value 1. This proof successfully mapped the Halting Problem directly into the domain of pure number theory.[14]

FRACTRAN is the ultimate formalization of this 1972 proof. The foundational FRACTRAN operation $T(n) = c_m n$ for $n \equiv m \pmod d$ is a pure Collatz-like generalized function, stripped of the additive $+1$ shift and relying purely on multiplicative rational coefficients.[4]

Ivan Korec's extensive, career-spanning research into the 3x+1 problem and its generalizations is critical to understanding the topological limits of these residue classes.[24] Korec (1992) demonstrated that the iterates of the Collatz function can be explicitly encoded into a one-dimensional nearest-neighbor cellular automaton utilizing only 7 discrete states.[4] By reducing the chaotic dynamics of the Collatz sequence to a 7-state cellular automaton, Korec established a tight, rigorous bound on the spatial complexity required for Collatz-like residue mixing. Korec also developed an 11-state cellular automaton capable of real-time generation of primes [26], further bridging the gap between discrete automata and number-theoretic sequences.

Furthermore, Korec meticulously analyzed the arithmetic density of Collatz sets. In his 1994 paper, "A Density Estimate for the 3x+1 Problem," Korec proved that for any value $\gamma > \frac{\log 3}{\log 4} \approx 0.7925$, the set of integers n that possess some iterate strictly less than n^γ has an asymptotic density of exactly one.[25]

This density estimate is highly relevant to establishing the FRACTRAN universal threshold. Functions that belong to Hasse's class are mathematically characterized as "mixing" on residue classes $\pmod{d^k}$ for all powers of k.[25] This mixing creates a pseudo-random dispersion of values across the modulo space, leading to the unpredictable trajectories seen in the Collatz conjecture. However, FRACTRAN functions that successfully encode Universal Turing Machines must exhibit specific, highly constrained *non-mixing* behaviors to preserve their deterministic data states (the prime register values) over unbounded computational iterations.[24]

Because a universal FRACTRAN program must preserve the strict mathematical integrity of the

prime exponents r_j while manipulating the state sequence, the underlying piecewise affine map cannot uniformly mix the residue classes.[25] The preservation of state requires the existence of a specific bounding constant C such that:

$$\left|F(n) - \frac{n}{d}\right| \leq C \quad \text{if } n \equiv 0 \pmod{d}$$

This bounding condition allows the FRACTRAN machine to maintain rigid, predictable state transitions without succumbing to catastrophic algebraic diffusion.[4]

Therefore, the absolute minimum number of residue classes modulo d must be large enough to geometrically partition the integer space into strictly non-mixing, independent functional domains. Korec's reduction of Universal Register Machines to exactly 14 instructions [13] perfectly parameterizes this necessary partition: the integer state space must be divided into at least 14 distinct linear functional domains to suppress Hasse-class mixing and maintain the coherent, uncorrupted execution of a Turing machine over infinite time steps.

Computational Complexity: Time vs. Space in Bi-Tag Systems

While Korec's 14-instruction Universal Register Machine provides a direct, irrefutable baseline for the minimum distinct residue classes modulo d [20], contemporary theoretical computer science research into small Universal Turing Machines offers parallel perspectives on the threshold. Turlough Neary and Damien Woods have extensively surveyed, analyzed, and discovered exceptionally small polynomial-time Universal Turing Machines.[18]

Neary and Woods established new minimal polynomial-time UTMs featuring extraordinarily small state-symbol pairs, specifically $(9, 3)$ and $(18, 2)$ configurations.[27] To map these highly optimized machines into the arithmetic space of FRACTRAN, they must be converted into counter machines. The standard, mathematically sound reduction pathway follows a specific trajectory: Turing Machine $\Longrightarrow$ Tag System / Bi-Tag System $\Longrightarrow$ 2-Counter Machine $\Longrightarrow$ FRACTRAN.[27]

A bi-tag system operates fundamentally differently from a Minsky machine. It functions on a dynamic queue of symbols, reading the head of the queue and appending newly generated symbols to the tail. Neary and Woods proved that bi-tag systems can efficiently simulate UTMs in polynomial time, entirely bypassing the exponential time slowdown that intrinsically plagues Minsky register machines.[27] Simulating a bi-tag system within the syntax of FRACTRAN requires the prime exponents to act as sliding dynamic queues rather than simple static

arithmetic counters.

However, mapping an $(18, 2)$ UTM via a bi-tag system into a functional FRACTRAN program significantly increases the total instruction count, generally yielding programs with upwards of 40 to 60 distinct fractions. Thus, while these machines are hyper-optimized for time complexity and algorithmic efficiency [27], they do not lower the absolute algebraic minimum threshold of residue classes modulo d. The optimization of time complexity—avoiding the exponential overhead of Minsky machines—mandates a massive expansion of the syntax complexity, requiring more fractions, a vastly enlarged modulus d, and a significantly broader prime signature.[18]

This establishes a fundamental, inverse mathematical relationship within the FRACTRAN threshold:

- To minimize the spatial complexity and reduce the number of distinct residue classes modulo d down to the absolute limit of 14, the computational system must accept exponential or hyper-exponential time complexity, which is the inescapable limit of the Minsky machine architecture.[20]
- To minimize the time complexity and achieve polynomial simulation speeds, the system must drastically increase the number of fractions, enlarging the modulus d and utilizing a highly complex prime signature.[18]

This computational explosion is not merely a theoretical artifact but a practical engineering constraint. The transition from a Turing Machine to a FRACTRAN program is a mapping from a discrete tape topology to a continuous arithmetic topology. In a traditional Turing Machine, the spatial complexity is represented by the physical length of the tape. In FRACTRAN, the spatial complexity is encapsulated entirely by the sheer magnitude of the integer N.[10] Because the registers are stored as exponents ($N = 2^{r_1} 3^{r_2} \cdots$), linearly incrementing a register scales the state integer exponentially. If a FRACTRAN program requires k steps to compute a function, the integer N can easily grow to the order of $\mathcal{O}(c^{2^k})$.[9]

This extreme scaling was highlighted during the 2014 British Informatics Olympiad, where students were tasked with implementing FRACTRAN interpreters.[10] The Olympiad materials explicitly noted that representing FRACTRAN states as raw numerical values would instantly cause standard integer overflow in modern computers.[16] To execute FRACTRAN efficiently, the interpreters must completely abandon the raw integer N and instead process the state as an array of prime factorizations.[16] This practical programming challenge reinforces the theoretical reality that FRACTRAN's power lies strictly in its prime exponents, and any attempt to deviate

from the $(1, 1, 1$ square-free prime signature threshold would cause the already hyper-exponential integer scaling to become completely mathematically intractable.[11]

The following table summarizes the comparative landscape of small universal machines and their trade-offs when mapped into FRACTRAN space.

Universal Paradigm	**Minimum States/Rules**	**Resulting FRACTRAN Complexity**	**Performance Characteristic**
Minsky 2-Counter (1967)	$\sim 40+$ Instructions	$\sim 80+$ Residue Classes	Exponential Time
Gregušová & Korec (1979)	37 Instructions	~ 74 Residue Classes	Exponential Time
Korec Minimal URM (1996)	14 Instructions	14 Distinct Residue Classes	Hyper-Exponential Time (Absolute Minimum Space)
Neary & Woods Bi-Tag UTM	(18, 2) State-Symbol	$\sim 40 - 60+$ Residue Classes	Polynomial Time (High Modulus Complexity)

Undecidability, Rice's Theorem, and Semi-Unification

Because FRACTRAN programs are completely computationally universal, all formal properties and theorems defining Turing completeness apply directly to the arithmetic sequences they generate. Conway explicitly leveraged the FRACTRAN syntax to definitively prove the undecidability of arbitrary Collatz-like piecewise affine problems.[14]

Rice's Theorem, a fundamental concept in computability theory, states that all non-trivial semantic properties of programs are mathematically undecidable.[14] A semantic property is any

property relating to the program's actual behavior or output, rather than its syntax.[14] In the context of the FRACTRAN threshold, Rice's theorem guarantees that there exists no general algorithm that can analyze an arbitrary prime signature and sequence of rational fractions to determine if the resulting sequence of integers $\{N_t\}$ will halt, fall into a continuous loop, or grow infinitely.[14]

This directly ties the minimal FRACTRAN threshold to the mathematical problem of Semi-Unification.[23] The problem of semi-unification—determining if there exist specific substitutions capable of satisfying a given set of term inequalities—has been constructively and rigorously reduced from the Halting Problem via small universal register machines, such as those formulated by Korec.[23] The minimal instruction sets, requiring only 14 rules, serve as the explicit, structural bridge used to construct undecidable instances of semi-unification.

If a mathematician were to analyze a FRACTRAN machine situated below the theoretical threshold—for instance, an arbitrary program consisting of only 5 fractions with a strictly constrained prime signature—the resulting sequences could theoretically be resolved by automated theorem provers. The topological structure of the residue classes modulo d for such a small machine would fundamentally lack the necessary branching logic and structural complexity to embed an unresolvable paradoxical loop, which is the prerequisite for the Halting contradiction.

The 14-fraction and 14-residue class threshold is therefore not just a syntactical curiosity or an arbitrary optimization milestone; it is the absolute mathematical boundary at which linear arithmetic transitions from decidable pattern generation into the chaotic domain of formal undecidability.[14] Below this threshold, arithmetic sequences are entirely predictable. At and above this threshold, arithmetic becomes capable of expressing every unresolvable paradox in the universe of computable numbers.

Furthermore, the mechanics of FRACTRAN and its minimal thresholds hold profound implications for emerging paradigms, including quantum computation. As noted in texts such as Nielsen and Chuang's foundational works on quantum information, universal computation relies on the ability to perfectly simulate any Turing machine.[33] FRACTRAN's reliance on reversible and irreversible multiplicative operators provides a unique lens through which to view quantum bit states, mapping continuous quantum amplitudes to the continuous scaling of integer spaces, constrained only by the absolute minimum residue classes and prime signature thresholds defined by Korec's parameters.[20]

Synthesizing the Minimal Computational Architecture

The synthesis of these disparate domains—number theory, topology, cellular automata, and computability theory—reveals the stark, unparalleled architectural efficiency of integer factorization as a universal memory medium. To visually conceptualize the absolute limits

required to encode an arbitrary computable sequence via Alan Turing's theoretical framework into a FRACTRAN system [1], we can define the final boundaries of the arithmetic state geometry.

Architectural Parameter	Absolute Minimum Threshold	Theoretical Basis and Proof Origin
Data Registers	2	Marvin Minsky's proof that a 2-Counter Machine is Turing-complete.[12]
Minimum Distinct Primes	16	2 primes for data counters, 14 One-Hot state primes required to maintain coefficient bounds based on Korec's limits.[20]
Max Coefficient Prime Exp.	1 (Square-free fractions)	The $(1, 1, 1$ Prime Signature limit; single-step arithmetic increments ensure deterministic non-ambiguous state traversal.[11]
Minimum Residue Classes	14	Mapped strictly from Ivan Korec's 14-Instruction Universal Register Machine, establishing exactly 14 distinct functional arithmetic branches.[13]
Collatz Modulus d **Parameter**	$\mathrm{lcm}(B_1 \ldots B_1$	The piecewise affine map requires modulo partitioning distinct enough to prevent Hasse-class uniform mixing and algebraic diffusion.[4]

This minimal architecture conclusively proves that the universe of computable numbers, and by extension the entirety of algorithmic logic [1], does not require a complex silicon processor, nor does it even require the finite tape and read/write head of a theoretical Turing machine. It requires nothing more than the unique prime factorization of a single integer, and a highly specific cascade of exactly 14 strictly governed rational coefficients operating within the rigid

mathematical parameters of modular residue classes.

The investigation into the FRACTRAN threshold isolates the exact mathematical moment at which pure arithmetic becomes Turing-complete. By synthesizing the research of John Conway, Marvin Minsky, Ivan Korec, and Turlough Neary, the absolute minimal parameters required for a universal FRACTRAN program are rigorously and undeniably bounded.

First, the absolute minimum number of distinct residue classes modulo d required to successfully encode a Universal Turing Machine is strictly mapped to the minimal instruction set of a Universal Register Machine. Through Ivan Korec's aggressive mathematical optimizations, this boundary is established at 14 distinct functional residue classes.[13] Below this threshold, the piecewise affine map modulo d lacks the necessary conditional branching complexity to simulate the memory tape and state control of a UTM without falling into predictable, unresolvable trivial loops.[23]

Second, the minimum prime signature required for the fractional coefficients a_i is definitively defined by the absolute upper exponent bound of 1, establishing the $(1, 1, 1)$ prime signature threshold.[11] Fractions forming the universal state transition map strictly require square-free numerators and denominators to ensure absolute determinism and prevent overlapping, chaotic arithmetic modifications to the discrete prime registers.[7] Furthermore, the underlying dimension of this prime signature requires a minimum of 16 distinct primes for a strict linear state mapping that matches the 14-instruction minimal bounds.[12]

These thresholds represent the absolute bedrock of computation. FRACTRAN demonstrates, with unyielding mathematical precision, that the Halting problem, Rice's Theorem, and the full extent of Turing universality are not exclusively properties of computer science or artificial automata. Rather, they are fundamentally, inextricably embedded within the arithmetic topology of the prime numbers and the non-mixing dynamics of modular residue classes.[4] Through this lens, computation is not an invention, but an inherent geometric property of the integers themselves.

Works cited

1. Universal Turing machine - Wikipedia, accessed March 12, 2026, https://en.wikipedia.org/wiki/Universal_Turing_machine
2. FRACTRAN - UNC Computer Science, accessed March 12, 2026, https://www.cs.unc.edu/~stotts/COMP210-s23/madMath/Conway87.pdf
3. An update on the 3x+ 1 problem - ResearchGate, accessed March 12, 2026, https://www.researchgate.net/publication/228990429_An_update_on_the_3x_1_problem
4. arXiv:math/0309224v13 [math.NT] 11 Jan 2011, accessed March 12, 2026,

https://arxiv.org/pdf/math/0309224
5. Fractran: A Ridiculous Logical Language - Berkeley Graduate Lectures, accessed March 12, 2026, https://gradlectures.berkeley.edu/lecture/fractran-a-ridiculous-logical-language/
6. Fractran: A Ridiculous Logical Language with John Conway [2012] - YouTube, accessed March 12, 2026, https://www.youtube.com/watch?v=548BH-YFT1E
7. FRACTRAN | Complex Projective 4-Space, accessed March 12, 2026, https://cp4space.hatsya.com/2013/05/12/fractran/
8. Fractran A Simple Programming Language - HKopp - koppian adventures, accessed March 12, 2026, https://hkopp.github.io/2018/03/fractran-a-simple-programming-language
9. FRACTRAN - Wikipedia, accessed March 12, 2026, https://en.wikipedia.org/wiki/FRACTRAN
10. Genius At Play, accessed March 12, 2026, https://www.essra.org.cn/upload/202006/132364148588881489.pdf
11. mathworld-titles.csv, accessed March 12, 2026, https://people.math.wisc.edu/~roch/mmids/utils/datasets/mathworld-titles.csv
12. Remembering John Conway's FRACTRAN, a ridiculous, yet surprisingly deep language, accessed March 12, 2026, https://raganwald.com/2020/05/03/fractran.html
13. Constructive Many-one Reduction from the Halting Problem to Semi-unification (Extended Version\rsuper*) - arXiv.org, accessed March 12, 2026, https://arxiv.org/html/2208.13428v5
14. Remembering John Conway's FRACTRAN, a ridiculous, yet surprisingly deep language : r/ProgrammingLanguages - Reddit, accessed March 12, 2026, https://www.reddit.com/r/ProgrammingLanguages/comments/ghpek3/remembering_john_conways_fractran_a_ridiculous/
15. math-kook-sample-chapters.pdf - Kevin Knight, accessed March 12, 2026, https://kevincrawfordknight.github.io/math-kook-sample-chapters.pdf
16. 2014 FRACTRAN - The British Informatics Olympiad, accessed March 12, 2026, https://www.olympiad.org.uk/papers/2014/final/fractran.pdf
17. Mathematica Slovaca - DML-CZ, accessed March 12, 2026, https://dml.cz/bitstream/handle/10338.dmlcz/136744/MathSlov_49-1999-1_13.pdf
18. Programmability of Chemical Reaction Networks - Caltech, accessed March 12, 2026, https://www.dna.caltech.edu/Papers/programmability_of_CRNs_preprint2008.pdf
19. Reachability games and related matrix and word problems - University of Liverpool Repository, accessed March 12, 2026, https://livrepository.liverpool.ac.uk/3018108/1/201052026_Feb2018.pdf
20. Spiking Neural P Systems with Autapses | Request PDF - ResearchGate, accessed March 12, 2026, https://www.researchgate.net/publication/351030558_Spiking_Neural_P_Systems_with_Autapses
21. Residue Class -- from Wolfram MathWorld, accessed March 12, 2026, https://mathworld.wolfram.com/ResidueClass.html

22. NOTICES OF THE AMERICAN MATHEMATICAL SOCIETY, accessed March 12, 2026, https://www.ams.org/journals/notices/196601/196601FullIssue.pdf
23. (PDF) Constructive Many-one Reduction from the Halting Problem to Semi-unification (Extended Version) - ResearchGate, accessed March 12, 2026, https://www.researchgate.net/publication/376405243_Constructive_Many-one_Reduction_from_the_Halting_Problem_to_Semi-unification_Extended_Version
24. arXiv:math/0309224v10 [math.NT] 10 Jun 2008, accessed March 12, 2026, https://www.arxiv.org/pdf/math/0309224v10
25. 1963 1999 | PDF | Teaching Mathematics - Scribd, accessed March 12, 2026, https://www.scribd.com/document/151748854/1963-1999
26. Real-Time Prime Generation with CA | PDF | Cartesian Coordinate System - Scribd, accessed March 12, 2026, https://www.scribd.com/document/842229132/KOREC-Real-Time-Generation-of-Primes-by-One-Dimensional-Cellular-Automaton-With-11-States
27. Tag systems and Collatz-like functions, accessed March 12, 2026, https://www.researchgate.net/publication/222409907_Tag_systems_and_Collatz-like_functions
28. Counterintuitive results for undergraduates - Theoretical Computer Science Stack Exchange, accessed March 12, 2026, https://cstheory.stackexchange.com/questions/31593/counterintuitive-results-for-undergraduates
29. Strong Computability and Variants of the Uniform Halting Problem, accessed March 12, 2026, https://www.semanticscholar.org/paper/0b809471a5ac9ec40ecc3a3371f325c88c5052fc
30. On simulating Turing machines with matrix semigroups with integrality tests - LJMU Research Online, accessed March 12, 2026, http://researchonline.ljmu.ac.uk/id/eprint/23352/1/On%20simulating%20Turing%20machines%20with%20matrix%20semigroups%20with%20integrality%20tests.pdf
31. BIO 2014 final - index - The British Informatics Olympiad, accessed March 12, 2026, https://www.olympiad.org.uk/papers/2014/final/round_two.html
32. Mathematics of Language - Linguistics - UCLA, accessed March 12, 2026, https://linguistics.ucla.edu/people/Kracht/courses/compling2-2007/formal.pdf
33. Quantum Computation and, accessed March 12, 2026, https://profmcruz.wordpress.com/wp-content/uploads/2017/08/quantum-computation-and-quantum-information-nielsen-chuang.pdf
34. Quantum computation and quantum information [2 ed.] 9781107002173 - DOKUMEN.PUB, accessed March 12, 2026, https://dokumen.pub/quantum-computation-and-quantum-information-2nbsped-9781107002173.html